# Case-Study zur Arbeitslosigkeit in Deutschland

*Teil 1: Daten beschaffen, bereinigen, prüfen*



Downloading package: dplyr

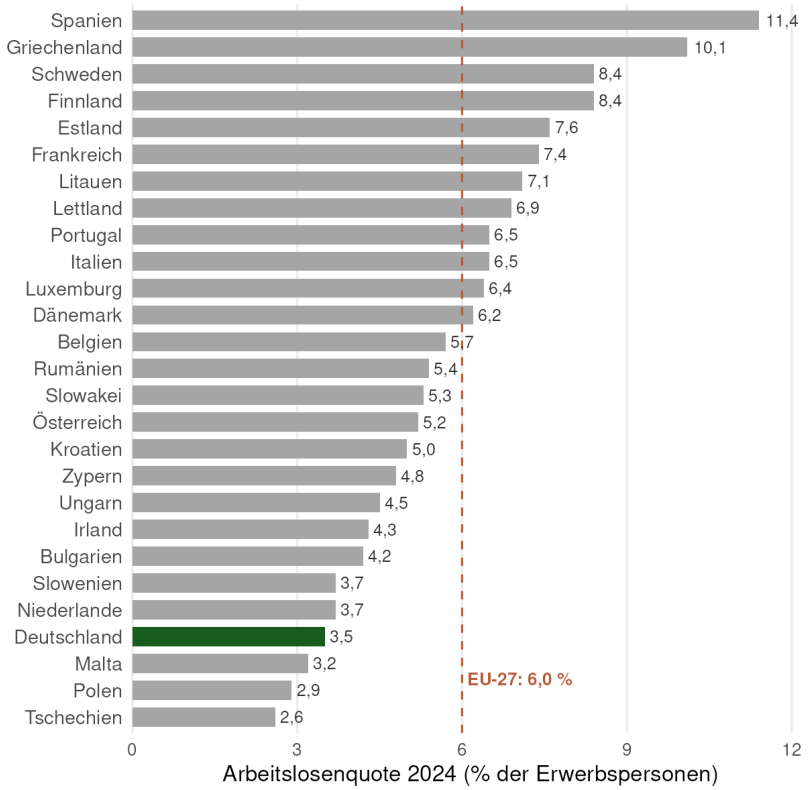
# Worum geht es?



Downloading package: dplyr

# Ziel der Case-Study

Deutschland hat europaweit eine der niedrigsten Arbeitslosenquoten:




> Doch gilt dies für alle Regionen in Deutschland?

> Warum ist die Arbeitslosenquote in manchen Regionen höher als in anderen?

> Dem gehen wir in dieser Case-Study auf den Grund.

Quelle: Eurostat, Arbeitslosenquote nach Ländern (`une_rt_a`), Jahresdurchschnitt 2024, 15–74 Jahre. Datenstand 10.09.2026.



Downloading package: dplyr

# Schätzfrage zum Einstieg

**30 Sekunden, Handzeichen:**

- Welches Bundesland hat die **höchste** Arbeitslosenquote?
- Welches die **niedrigste**?
- Wo liegt Baden-Württemberg?

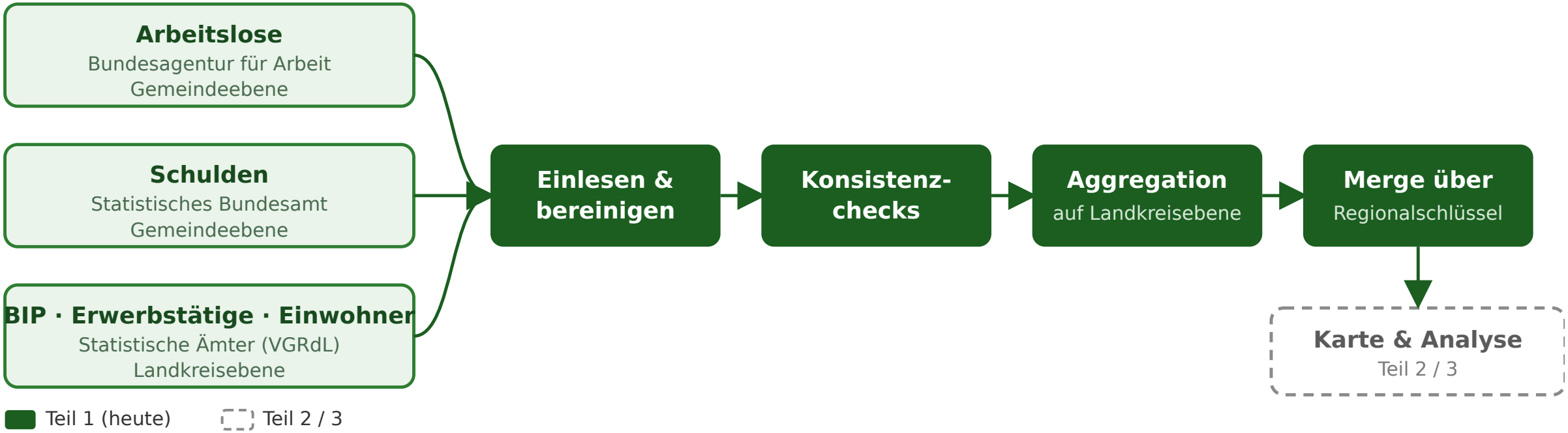
Auflösung am Ende des Abschnitts zu den Arbeitslosendaten.



Downloading package: tidyr

# Wo wollen wir hin?

Am Ende steht eine Karte: Arbeitslosigkeit, Verschuldung und Wirtschaftskraft **pro Landkreis**. Dafür bauen wir eine Daten-Pipeline:




**Teil 1 = alles Grüne.** Unspektakulär, aber hier passieren 80 % der Fehler in echten Projekten.



Downloading package: tidyr

# Ziele der Case-Study

Die Case-Study besteht aus **mehreren Teilen** und begleitet die komplette Vorlesung als **konkretes Anschauungsobjekt**. Sie dient als

- konkretes und umfangreiches Beispiel für ein Projekt,
- Quelle ökonomischer und geographischer Kenntnisse über Deutschland,
- Beispiel, wie Statistik und Programmierung in der empirischen Arbeit zusammenspielen.

**Das eigentliche Lernziel von Teil 1**

Sie sollen **Code lesen, prüfen und reparieren** können, egal ob er von Ihnen, von Kommiliton:innen oder von Chat AI stammt. Deshalb gibt es regelmäßig 🐛 **Bug Hunts**: Code, der läuft, aber falsch ist.



Downloading package: tidyr

# Drei Arten von Fehlern

| | Was passiert? | Wie merkt man es? | Gefahr |
|---|---|---|---|
| **Fehler** (`Error`) | Code stoppt | sofort | gering |
| **Warnung** (`Warning`) | Code läuft weiter | nur wenn man liest | mittel |
| **Stiller Fehler** | Code läuft, Ergebnis falsch | nur durch **Prüfen** | **hoch** |

KI-generierter Code produziert fast nie Fehler der ersten Art, aber regelmäßig der dritten. Alle Bug Hunts heute sind stille Fehler.

**Was wir dagegen setzen ist der Review-Loop:** Verdacht → Prüfung (`nrow()`, `glimpse()`, `count()`, externe Quelle) → Fix.



Downloading package: stringr

# So läuft die Vorlesung

🔮 **Prognose**: Sie tippen, bevor der Output kommt (Handzeichen).

🐛 **Bug Hunt**: KI-Code, der läuft, aber falsch ist. 1–2 Minuten zu zweit.

✍️ **Jetzt Sie**: kurze Programmieraufgabe im eigenen RStudio.

📱 **Kahoot-Checkpoint**: am Ende jedes Abschnitts, 6–7 Fragen, 5 Minuten. Die Bug Hunts kommen dort wieder vor.

Halten Sie **kahoot.it** auf dem Handy bereit; die PIN steht jeweils auf der Folie.

ⓘ Bei den Kahoot-Quizzes gibt es immer etwas zu gewinnen.



Downloading package: stringr

# Zwei Umgebungen zum Coden

### Direkt auf der Folie (webR)

Code-Felder mit *Run Code* laufen **in Ihrem Browser**; nichts installieren, nichts hochladen. Für die kurzen ✍️-Aufgaben und die Bug-Hunt-Fixes.

- Hinweise und Lösung lassen sich aufklappen
- Ein automatischer Check sagt Ihnen, ob es stimmt
- Beim ersten Öffnen lädt webR ca. 20 Sekunden

Probieren Sie es gleich aus:

R Code | Start Over | ▷ Run Code

```
1 x <- c(3, 1, 4, 1, 5)
2 sum(x) / length(x)
```

### In RStudio (Ihr Rechner oder Posit Cloud)

Für die 20-Minuten-Aufgaben mit den vollständigen Excel-Dateien aus dem Repository, so, wie Sie später auch im Projekt arbeiten.

- GitHub Desktop → Pull → `case-study/` öffnen
- Chat AI erlaubt, Prüfen Pflicht

> Die Felder auf den Folien speichern nichts. Was Sie behalten wollen, kopieren Sie in Ihr eigenes R-Skript.



Downloading package: readr

# Datensätze beschaffen



Downloading package: readr

# Teil 1 im Überblick

- Daten einlesen
- Daten bereinigen und in eine geeignete Form bringen (`tidy`)
- Daten **prüfen** (intern und gegen externe Quellen)

Anwenden auf

- Daten zur Arbeitslosenstatistik
- Daten zur Verschuldung einzelner Landkreise bzw. Gemeinden
- Daten zum BIP



Wir nutzen die Base-R-Pipe `|>`. Wenn Chat AI Ihnen `%>%` liefert: funktioniert genauso.



Downloading package: ggplot2

# Wichtig für die Datenbeschaffung

- Zuverlässige, qualitativ hochwertige Datenquellen finden (**Beispiel: Warum man BIP-Daten diktatorisch regierter Staaten nicht trauen kann**)
- Automatisierten Download programmieren
- Einlesen, verarbeiten und zusammenführen verschiedener Datensätze in R

Verbindung zum 2. RTutor Problem Set:

- **Im Problem Set:** Kennzahlen zu verschiedenen Ländern der EU
- **Hier:** Kennzahlen innerhalb Deutschlands

In Case-Study und Problem Sets treffen Sie auf konkrete Probleme, die Sie mit Ihren Kenntnissen aus der Vorlesung lösen sollen.



Downloading package: ggplot2

# Daten beschaffen

Woher beziehen wir unsere Informationen?

- Verschuldung der **Gemeinden**: Statistisches Bundesamt, Integrierte Schulden der Gemeinden und Gemeindeverbände
- Arbeitslosigkeit auf **Verwaltungsgemeinschaftsebene**: Bundesagentur für Arbeit
- BIP auf **Landkreisebene**: Statistische Ämter des Bundes und der Länder

> Zuverlässige und qualitativ hochwertige Datenquellen ausfindig machen ✔

# Daten herunterladen

- Daten können von URLs mit `readxl` und `readr` direkt eingelesen werden
- Allerdings: Wenn die URL nicht mehr verfügbar ist, was dann?
  - → Daten immer mit `download.file()` herunterladen und in `data/` abspeichern!

Automatisierten Download programmieren (in der ausformulierten Case-Study) ✔

Die Daten liegen bereits im GitHub-Repository “case-study-germany”. Klonen Sie dieses Repository auf Ihren PC!

# Klonen Sie unsere GitHub-Seite

- GitHub-Seite des Projektkurses → grüner “Code”-Button
- HTTPS-URL kopieren
- GitHub Desktop → “Clone a repository” → “URL” → einfügen

## Step-by-Step-Anleitung

Zu Beginn jeder Woche in GitHub Desktop auf “Pull” klicken: Folien, Case-Study und Tutorials werden automatisch aktualisiert.

05:00

# Nötige Pakete laden

```r
library(readxl)
library(skimr)
library(tidyverse)
```

```
── Attaching core tidyverse packages ──────────────────────────────────── tidyverse 2.0.0 ──
✔ dplyr     1.2.1     ✔ readr     2.2.0
✔ forcats   1.0.1     ✔ stringr   1.6.0
✔ ggplot2   4.0.3     ✔ tibble    3.3.1
✔ lubridate 1.9.5     ✔ tidyr     1.3.2
✔ purrr     1.2.2
── Conflicts ────────────────────────────────────────────────────── tidyverse_conflicts() ──
✖ dplyr::filter() masks stats::filter()
✖ dplyr::lag()    masks stats::lag()
ℹ Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become errors
```

🔮 **Prognose:** Nach `library(tidyverse)` rufen Sie `filter()` auf – welche Funktion wird genommen? Warum steht das hier?

# Arbeitslose

**Arbeitslose** → Schulden → BIP → Merge → Karte

**Nach diesem Abschnitt können Sie …**

- eine unschöne Excel-Tabelle gezielt einlesen (`sheet`, `skip`),
- mit `str_extract()` / `str_remove()` Informationen aus Text ziehen,
- Daten intern und gegen eine externe Quelle auf Konsistenz prüfen.

# Bevor wir Code schreiben: die Rohdaten

**3 Minuten, zu zweit:** Öffnen Sie im geklonten Repository `case-study/data/Arbeitslose_2024.xlsx.zip` (Blatt “Gesamt”). Notieren Sie **drei Dinge**, die das Einlesen in R erschweren werden.

Bundesagentur für Arbeit
Statistik

**Bestand an Arbeitslosen - Gesamt**

Länder, Regierungsbezirke, Kreise und Gemeinden (Gebietsstand = Datenstand)
Zeitreihe, Datenstand: Dezember 2024

**Rechtskreis Insgesamt**

Aus Datenschutzgründen und Gründen der statistischen Geheimhaltung werden Zahlenwerte von 1 oder 2 und Daten, aus denen rechnerisch auf einen solchen Z

zurück zum Inhalt

| | Jahresdurchschnitte | | Rechtskreis Insgesamt - Bestand an Arbeitslosen - Gesamt | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|
| | Jahresdurch-schnitt | Jahresdurch-schnitt | Januar 2023 | Februar 2023 | März 2023 | April 2023 | Mai 2023 | Juni 2023 | Juli 2023 | August 2023 |
| Region | 2023 | 2024 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 |
| **Deutschland** | **2.608.672** | **2.787.112** | **2.616.019** | **2.620.169** | **2.593.774** | **2.585.677** | **2.543.743** | **2.554.982** | **2.617.192** | **2.695.827** |
| **01 Schleswig-Holstein** | **87.757** | **92.493** | **89.641** | **89.838** | **88.511** | **87.613** | **85.710** | **85.291** | **87.184** | **90.207** |
| **01001 Flensburg, Stadt** | **4.131** | **4.417** | **4.241** | **4.196** | **4.105** | **4.115** | **4.039** | **3.953** | **4.119** | **4.315** |
| **01002 Kiel, Landeshauptstadt** | **10.581** | **11.137** | **10.500** | **10.411** | **10.454** | **10.746** | **10.410** | **10.464** | **10.697** | **11.134** |
| **01003 Lübeck, Hansestadt** | **9.216** | **9.596** | **9.451** | **9.405** | **9.348** | **9.355** | **9.131** | **9.064** | **9.195** | **9.448** |
| **01004 Neumünster, Stadt** | **3.604** | **3.771** | **3.525** | **3.585** | **3.586** | **3.646** | **3.620** | **3.594** | **3.654** | **3.707** |
| **01051 Dithmarschen** | **4.126** | **4.360** | **4.360** | **4.397** | **4.290** | **4.095** | **3.954** | **3.988** | **4.111** | **4.180** |
| **01053 Herzogtum Lauenburg** | **5.587** | **5.722** | **5.743** | **5.733** | **5.671** | **5.612** | **5.412** | **5.346** | **5.494** | **5.858** |
| **01054 Nordfriesland** | **4.491** | **4.677** | **5.314** | **5.336** | **4.982** | **4.278** | **4.031** | **4.013** | **4.138** | **4.220** |
| **01055 Ostholstein** | **5.177** | **5.356** | **6.053** | **6.042** | **5.624** | **4.969** | **4.755** | **4.721** | **4.776** | **4.980** |
| **01056 Pinneberg** | **9.280** | **9.962** | **9.308** | **9.331** | **9.106** | **9.156** | **9.130** | **9.068** | **9.190** | **9.501** |
| **01057 Plön** | **2.848** | **2.970** | **2.869** | **2.925** | **2.912** | **2.858** | **2.802** | **2.782** | **2.793** | **2.964** |
| **01058 Rendsburg-Eckernförde** | **6.303** | **6.680** | **6.221** | **6.175** | **6.277** | **6.477** | **6.346** | **6.292** | **6.396** | **6.630** |
| **01059 Schleswig-Flensburg** | **5.573** | **5.925** | **5.599** | **5.691** | **5.556** | **5.521** | **5.446** | **5.445** | **5.536** | **5.742** |
| **01060 Segeberg** | **7.645** | **8.080** | **7.540** | **7.604** | **7.526** | **7.565** | **7.513** | **7.502** | **7.695** | **7.890** |
| **01061 Steinburg** | **4.113** | **4.176** | **4.016** | **4.077** | **4.132** | **4.156** | **4.125** | **4.145** | **4.246** | **4.341** |
| **01062 Stormarn** | **5.083** | **5.666** | **4.901** | **4.930** | **4.942** | **5.064** | **4.996** | **4.914** | **5.144** | **5.297** |
| **02 Hamburg** | **80.806** | **88.471** | **78.809** | **78.740** | **78.224** | **78.445** | **78.603** | **79.457** | **81.999** | **84.434** |
| **02000 Hamburg, Freie und Hansestadt** | **80.806** | **88.471** | **78.809** | **78.740** | **78.224** | **78.445** | **78.603** | **79.457** | **81.999** | **84.434** |
| **03 Niedersachsen** | **251.873** | **263.000** | **252.401** | **254.469** | **250.127** | **250.171** | **245.620** | **246.145** | **255.020** | **260.767** |

03:00

# Daten einlesen

Unterschiedliche Dateien und Tabellenblätter – was verwenden wir?

Zuerst legen wir den Pfad zum Datenordner fest. Im RStudio-Projekt `case-study/` liegen die Rohdaten im Unterordner `data/`:

```
datenpfad <- "data"
```

```
# Im ZIP liegen zwei Tabellen; uns interessiert die Anzahl der Arbeitslosen ([1])
alo_zip  <- file.path(datenpfad, "Arbeitslose_2024.xlsx.zip")
alo_name <- as.character(unzip(alo_zip, list = TRUE)$Name)
alo_name <- alo_name[1]
unzip(alo_zip, alo_name)
```

**Vermutung:** Über das Tabellenblatt “Inhaltsverzeichnis” werden wir schlauer.

# Daten einlesen

```r
alo_inhalt <- read_xlsx(alo_name, sheet = "Inhaltsverzeichnis")
head(alo_inhalt, 7)
```

```
# A tibble: 7 × 1
  Inhaltsverzeichnis
  <chr>
1 <NA>
2 <NA>
3 Arbeitslose - Zeitreihe
4 <NA>
# ℹ 3 more rows
```



Bundesagentur für Arbeit
Statistik

**Inhaltsverzeichnis**

**Arbeitslose - Zeitreihe**

**Tabelle**
**Bestand an Arbeitslosen**

| |
|---|
| **Kreiszusammenfassung** |
| Übersicht nach Kreisen |
| |
| Insgesamt |
| Rechtskreis |
| SGB III |
| SGB II |
| Geschlecht |
| Männer |
| Frauen |
| Staatsangehörigkeit |
| Deutsche |
| Ausländer |
| ausgewählte Personengruppen |
| 15 bis unter 25 Jahre |
| 55 Jahre und älter |
| 55 bis unter 65 Jahre |
| Langzeitarbeitslose |
| schwerbehinderte Menschen |
| Methodische Hinweise und Infoseite |
| Hinweis zu Arbeitslosigkeit |
| Info |

**Alternative:** Excel-Datei in Excel/LibreOffice öffnen und dann entscheiden, welches Blatt Sie einlesen.

# Welche Information brauchen wir?

- **Die Anzahl aller Arbeitslosen pro Gemeinde (SGB II und III) aus dem Jahr 2024**
- … pro Gemeinde für einen bestimmten Rechtskreis (z.B. nur SGB II)
- … pro Gemeinde für Rechtskreis und Alter (z.B. SGB II, unter 25)

> Was ist hier eine **Beobachtung**?

Dazu brauchen wir noch Regionalschlüssel und Gemeindenamen.

- Einfachster Weg: die ersten acht Zeilen abschneiden (`skip = 8`) …
- … und nur die ersten 3 Spalten behalten.

# Einlesen und bereinigen

> 🔮 **Prognose:** Wie viele Zeilen hat `alo_skip` – mehr oder weniger als die ca. 10.800 Gemeinden in Deutschland?

```r
alo_skip <- read_xlsx(alo_name, sheet = "Gesamt", skip = 8)

data_alo <- alo_skip |>
  select(c(`...1`, Jahresdurchschnitte, `...3`)) |>          # nur die ersten 3 Spalten
  mutate(Regionalschluessel = str_extract(`...1`, "[[:digit:]]+"),
         Gemeinde           = str_extract(`...1`, "[A-Z].*")) |>
  mutate(alo = as.numeric(`...3`)) |>                        # Arbeitslose als Zahl
  select(-c(`...1`, Jahresdurchschnitte, `...3`)) |>
  filter(!is.na(alo))                                         # Leer- und Textzeilen raus
```

# Schritt für Schritt durch die Pipe

Klicken Sie sich durch die Reiter – nach jedem `|>` sehen Sie, was mit den Daten passiert:

1 · `select()` | 2 · `mutate()` zerlegen | 3 · `mutate()` als Zahl | 4 · aufräumen

Von den 27 eingelesenen Spalten nur die **drei relevanten** behalten:

```
alo_skip |>
  select(c(`...1`, Jahresdurchschnitte, `...3`)) |>
  head(4)
```

```
# A tibble: 4 × 3
  ...1                 Jahresdurchschnitte ...3
  <chr>                <chr>               <chr>
1 <NA>                 Jahresdurch-schnitt Jahresdurch-schnitt
2 Region               2023                2024
3 Deutschland          2608671.5           2787112.0833333335
4 01 Schleswig-Holstein 87757.333333333328  92492.916666666672
```

# Einlesen und bereinigen: Ergebnis

```r
dim(alo_skip)
```

```
[1] 11198    27
```

```r
dim(data_alo)
```

```
[1] 11167     3
```

```r
head(data_alo, 4)
```

```
# A tibble: 4 × 3
  Regionalschluessel Gemeinde                 alo
  <chr>              <chr>                  <dbl>
1 01                 Schleswig-Holstein     92493.
2 01001              Flensburg, Stadt        4416.
3 01001000           Flensburg, Stadt        4416.
4 01002              Kiel, Landeshauptstadt 11137.
```

# 🐛 Bug Hunt 1: Läuft, aber …

Der Code von eben läuft ohne Fehler und ohne Warnung. Trotzdem steckt ein **stiller Fehler** in dieser Zeile:

```
1 Gemeinde = str_extract(`...1`, "[A-Z].*")
```

**2 Minuten, zu zweit:** Welche Gemeindenamen werden falsch extrahiert? Wie würden Sie das **prüfen**, ohne 10.000 Zeilen anzuschauen?

02:00

# 🐛 Bug Hunt 1: Prüfung

Gezielt nach **Randfällen** suchen (Namen, die mit Umlaut beginnen) und Rohdaten mit Ergebnis vergleichen:

```
alo_skip |>
  filter(str_detect(`...1`, "^[[:digit:]]+\\s+[ÄÖÜ]")) |>
  transmute(roh = `...1`, Gemeinde = str_extract(`...1`, "[A-Z].*"))
```

```
# A tibble: 22 × 2
  roh                           Gemeinde
  <chr>                         <chr>
1 05370028 Übach-Palenberg, Stadt Palenberg, Stadt
2 07132087 Ölsen                <NA>
3 07143060 Ötzingen             <NA>
4 07231125 Ürzig                <NA>
# ℹ 18 more rows
```

**Warum?** `[A-Z]` erfasst nur `A` bis `Z` – keine Umlaute. Bei “Überlingen, Stadt” springt die Regex zum ersten “echten” Großbuchstaben: “Stadt”.

# ✍ Jetzt Sie: Bug Hunt 1 reparieren

`alo_roh` enthält 40 Zeilen der Rohspalte (`roh`), darunter die Umlaut-Fälle. Ersetzen Sie die Regex so, dass `Gemeinde` immer der **ganze Name** ist. Die letzte Zeile soll das Ergebnis ausgeben.

Exercise | Start Over | Show Hint | ▷ Run Code

```
1 alo_roh |>
2   mutate(Gemeinde = str_extract(roh, "[A-Z].*")) |>
3   filter(str_detect(roh, "^[[:digit:]]+\\s+[ÄÖÜ]"))
```

# Bug Hunt 1: Fix

Nicht beschreiben, was wir *wollen*, sondern entfernen, was wir *nicht* wollen (Ziffern und Leerzeichen am Anfang):

```r
data_alo <- alo_skip |>
  mutate(Gemeinde = str_remove(`...1`, "^[[:digit:]]+\\s*"),
         Regionalschluessel = str_extract(`...1`, "[[:digit:]]+"),
         alo = as.numeric(`...3`)) |>
  select(Regionalschluessel, Gemeinde, alo) |>
  filter(!is.na(alo))
data_alo <- data_alo[-c(1, 2), ]

data_alo |> filter(str_detect(Gemeinde, "^[ÄÖÜ]")) |> head(3)
```

```
# A tibble: 3 × 3
  Regionalschluessel Gemeinde                alo
  <chr>              <chr>                 <dbl>
1 05370028           Übach-Palenberg, Stadt 1024
2 07132087           Ölsen                   2.58
3 07143060           Ötzingen               26.6
```

Eine Regex, die auf den ersten fünf Zeilen funktioniert, ist noch nicht getestet. Suchen Sie die Randfälle: Umlaute, Bindestriche, Sonderzeichen.

# Konsistenzcheck

- Machen die Angaben Sinn, sind die Daten in sich konsistent?
- Externe Datenquelle suchen und intern auf Konsistenz prüfen
- Informationen aggregieren und mit anderen Quellen vergleichen

- Zunächst: Arbeitslose je **Bundesland** 2024 → zweistelliger `Regionalschluessel`
  → Zeichen zählen mit `nchar()` (number of characters)
- **Externe Quelle:** Arbeitslose 2024 nach Ländern (Arbeitsagentur), Tabellenblatt 8

> 🔮 **Prognose:** Wie viele Zeilen liefert `filter(nchar(Regionalschluessel) == 2)`?

# Konsistenzcheck: externe Quelle

```r
check_alo_bundesland <- data_alo |>
  filter(nchar(Regionalschluessel) == 2) |>
  rename(bundesland = Regionalschluessel)
nrow(check_alo_bundesland)
```

```
[1] 16
```

**Wette (Handzeichen):** Stimmen unsere 16 Zahlen mit der Tabelle der Arbeitsagentur überein? **(a)** exakt **(b)** kleine Abweichungen **(c)** große Abweichungen

# Konsistenzcheck: externe Quelle

```r
print(check_alo_bundesland, n = 16)
```

```
# A tibble: 16 × 3
   bundesland Gemeinde                  alo
   <chr>      <chr>                   <dbl>
 1 01         Schleswig-Holstein      92493.
 2 02         Hamburg                 88471.
 3 03         Niedersachsen          263090.
 4 04         Bremen                  41116.
 5 05         Nordrhein-Westfalen    749712.
 6 06         Hessen                 194912.
 7 07         Rheinland-Pfalz        120552.
 8 08         Baden-Württemberg      269990
 9 09         Bayern                 285234.
10 10         Saarland                37241.
11 11         Berlin                 203242
12 12         Brandenburg             82795.
13 13         Mecklenburg-Vorpommern  64944.
14 14         Sachsen                139751.
15 15         Sachsen-Anhalt          84801.
16 16         Thüringen               68768.
```

zurück zum Inhalt

| Region | Jahresdurchschnitte<br>Jahresdurch-schnitt<br>2024 |
|---|---|
| **Deutschland** | **2.787.112** |
| **01 Schleswig-Holstein** | **92.493** |
| **02 Hamburg** | **88.471** |
| **03 Niedersachsen** | **263.090** |
| **04 Bremen** | **41.116** |
| **05 Nordrhein-Westfalen** | **749.712** |
| **06 Hessen** | **194.912** |
| **07 Rheinland-Pfalz** | **120.552** |
| **08 Baden-Württemberg** | **269.990** |
| **09 Bayern** | **285.234** |
| **10 Saarland** | **37.241** |
| **11 Berlin** | **203.242** |
| **12 Brandenburg** | **82.795** |
| **13 Mecklenburg-Vorpommern** | **64.944** |
| **14 Sachsen** | **139.751** |
| **15 Sachsen-Anhalt** | **84.801** |
| **16 Thüringen** | **68.768** |

Beide Datenreihen sind identisch.

# Interne Konsistenz prüfen

Berechne: Arbeitslose je Bundesland als **Summe** über die Gemeinden – und vergleiche mit der Arbeitsagentur.

```r
# Nur Gemeinden (8-stelliger Schlüssel), dann Landkreis und Bundesland ableiten
alo_meta <- data_alo |>
  filter(nchar(Regionalschluessel) == 8) |>
  mutate(landkreis  = str_extract(Regionalschluessel, "^.{5}"),
         bundesland = str_extract(Regionalschluessel, "^.{2}"))

alo_bundesland <- alo_meta |>
  group_by(bundesland) |>
  summarise(total_alo = sum(alo))

alo_landkreis <- alo_meta |>
  group_by(landkreis) |>
  summarise(total_alo = sum(alo)) |>
  rename(Regionalschluessel = landkreis)
```

# Interne Konsistenz prüfen

- `check_alo_bundesland`: von der Arbeitsagentur **bereits aggregiert**
- `alo_bundesland`: **unsere Summe** aus den Gemeinden

```r
check_consistency <- left_join(check_alo_bundesland, alo_bundesland, by = "bundesland") |>
  mutate(diff = alo - total_alo)
summary(check_consistency$diff)
```

```
      Min.    1st Qu.     Median       Mean    3rd Qu.       Max. 
-7.276e-12  0.000e+00  0.000e+00 -4.547e-13  0.000e+00  0.000e+00 
```

Es bestehen keine Unstimmigkeiten.

# Bug Hunt 2: Die KI-Variante

Chat AI schlägt für denselben Schritt vor:

```r
alo_bundesland_ki <- data_alo |>
  filter(nchar(Regionalschluessel) >= 5) |>          # "alle regionalen Einheiten"
  mutate(bundesland = str_sub(Regionalschluessel, 1, 2)) |>
  group_by(bundesland) |>
  summarise(total_alo = sum(alo))
```

**2 Minuten:** Läuft fehlerfrei. In welcher Zeile steckt der Fehler?

02:00

# Bug Hunt 2: Prüfung

```r
data_alo |>
  filter(nchar(Regionalschluessel) >= 5) |>
  mutate(bundesland = str_sub(Regionalschluessel, 1, 2)) |>
  group_by(bundesland) |>
  summarise(total_alo = sum(alo)) |>
  left_join(check_alo_bundesland, by = "bundesland") |>
  mutate(faktor = total_alo / alo)
```

```
# A tibble: 16 × 5
  bundesland total_alo Gemeinde              alo faktor
  <chr>          <dbl> <chr>               <dbl>  <dbl>
1 01           184986. Schleswig-Holstein  92493.      2
2 02           176943. Hamburg             88471.      2
3 03           526179. Niedersachsen      263090.      2
4 04            82231. Bremen              41116.      2
# ℹ 12 more rows
```

**Doppelzählung:** Landkreise (5 Stellen) *und* ihre Gemeinden (8 Stellen) werden addiert. Der Check gegen die externe Quelle findet das sofort – deshalb machen wir ihn.

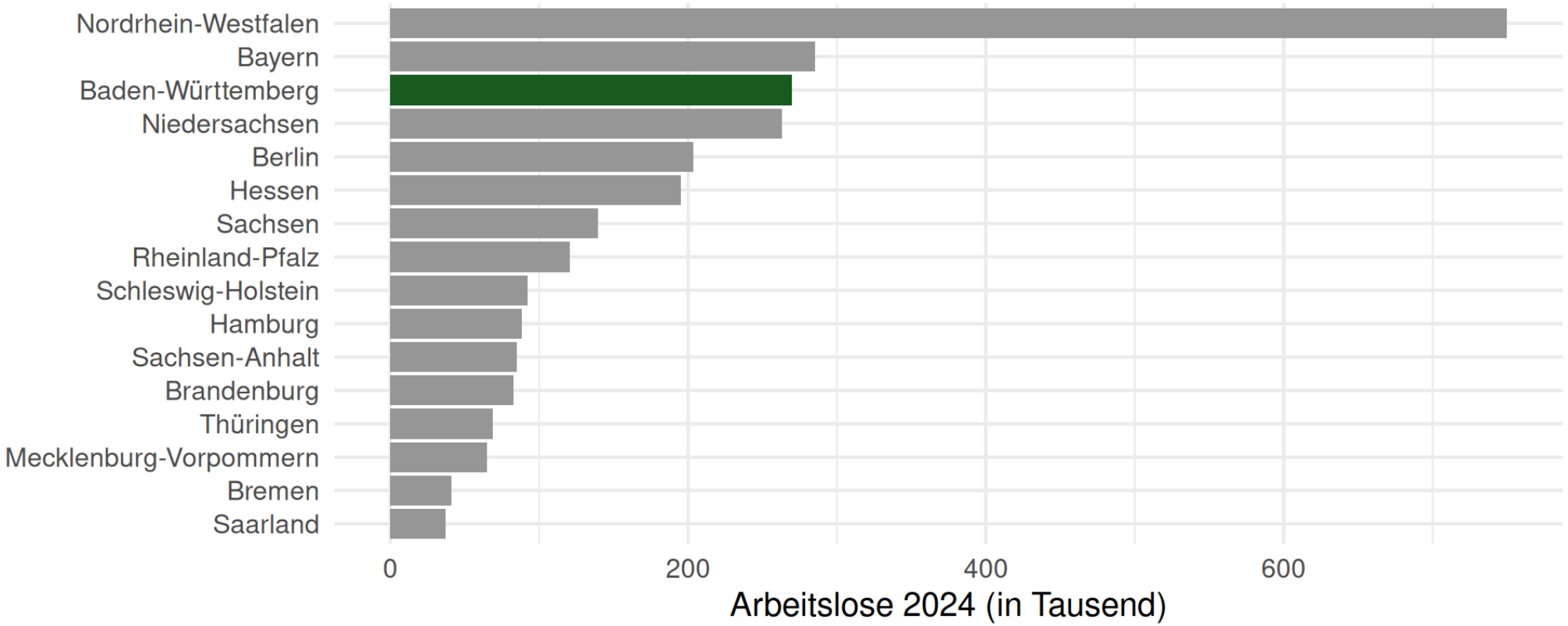
# ✍️ Jetzt Sie: Bug Hunt 2 reparieren

Reparieren Sie den KI-Code so, dass `diff` für alle 16 Bundesländer 0 ist. `data_alo` und `check_alo_bundesland` sind geladen.

Exercise | Start Over | Show Hint | ▷ Run Code

```
1 data_alo |>
2   filter(nchar(Regionalschluessel) >= 5) |>
3   mutate(bundesland = str_sub(Regionalschluessel, 1, 2)) |>
4   group_by(bundesland) |>
5   summarise(total_alo = sum(alo)) |>
6   left_join(check_alo_bundesland, by = "bundesland") |>
7   mutate(diff = alo - total_alo)
```

# Erster Blick auf die Daten




NRW führt klar. Beantwortet das unsere Schätzfrage? **Was fehlt**, um Regionen fair zu vergleichen?

# Kahoot-Checkpoint 1: Arbeitslose

**kahoot.it** · PIN am Beamer

## 7 Fragen, 5 Minuten.

- `read_xlsx()`, `skip`, `nchar()`
- Regionalschlüssel: 2 / 5 / 8 Stellen
- Bug Hunt 1: Umlaute und `[A-Z]`
- Bug Hunt 2: Doppelzählung
- Stille Fehler erkennen

Kahoot öffnen →

05:00

# Pro-Kopf-Verschuldung

Arbeitslose ✔ → **Schulden** → BIP → Merge → Karte

**Nach diesem Abschnitt können Sie …**

- viele gleich aufgebaute Tabellenblätter mit `map()` einlesen,
- Zahlen, die als Text gespeichert sind (`"{15 653}"`), sauber umwandeln,
- erklären, warum ein Mittelwert von Pro-Kopf-Werten meistens falsch ist.

# Schätzfrage: Ihr Landkreis

**Schätzen Sie (notieren!):** Kommunale Schulden **pro Kopf** in

- der Stadt Ulm?
- dem Alb-Donau-Kreis?
- dem Landkreis Biberach?

Ist Ulm stärker oder schwächer verschuldet als der Durchschnitt aller Gemeinden?

Auflösung am Ende des Abschnitts.

01:00

# Pro-Kopf-Verschuldung auf Gemeindeebene

- Gemeindeebene, Jahr 2024, Querschnittsdaten
- Vom Statistischen Bundesamt direkt als Excel-Tabelle heruntergeladen ✔

## Welche Tabellenblätter sollten wir nutzen?

```
1 schulden_pfad <- file.path(datenpfad, "Schulden_2024.xlsx")
2 excel_sheets(schulden_pfad)
```

```
 [1] "Titel"              "Impressum"          "Inhalt"             "Abkürzungen"
 [5] "Erläuterungen"      "Übersicht"          "SH"                 "NI"
 [9] "NW"                 "HE"                 "RP"                 "BW"
[13] "BY"                 "SL"                 "BB"                 "MV"
[17] "SN"                 "ST"                 "TH"                 "Statistische Ämter"
```

# Mehrere Tabellenblätter einlesen

- Nicht alle Informationen stehen in **einem** Tabellenblatt
  - → ein Blatt je Bundesland
  - → dieselbe Operation muss über mehrere Blätter laufen

> Zuerst an einem Beispiel schauen, welche Informationen wir benötigen:

```r
sh <- read_xlsx(schulden_pfad, sheet = "SH")
head(sh, 8)
```

```
# A tibble: 8 × 21
  Zurück zum In…¹ ...2  ...3  ...4  ...5  ...6  ...7  ...8  ...9  ...10 ...11 ...12 ...13 ...14
  <chr>           <chr> <chr> <chr> <chr> <chr> <chr> <chr> <chr> <chr> <chr> <chr> <chr> <chr>
1 <NA>            <NA>  <NA>  <NA>  <NA>  <NA>  <NA>  <NA>  <NA>  <NA>  <NA>  <NA>  <NA>  <NA>
2 <NA>            <NA>  <NA>  <NA>  <NA>  <NA>  <NA>  <NA>  <NA>  <NA>  <NA>  <NA>  <NA>  <NA>
3 Tabelle 1:    … <NA>  <NA>  <NA>  <NA>  <NA>  <NA>  <NA>  <NA>  <NA>  <NA>  <NA>  <NA>  <NA>
4 nach Höhe der … <NA>  <NA>  <NA>  <NA>  <NA>  <NA>  <NA>  <NA>  <NA>  <NA>  <NA>  <NA>  <NA>
# ℹ 4 more rows
# ℹ abbreviated name: ¹`Zurück zum Inhalt...1`
# ℹ 7 more variables: ...15 <chr>, ...16 <chr>, ...17 <chr>, ...18 <chr>, ...19 <chr>,
#   `Zurück zum Inhalt...20` <chr>, ...21 <chr>
```

# Was wir brauchen

- Regionalschlüssel, Gemeindename
- Einwohner
- Schulden des öffentlichen Bereichs insgesamt
- Schulden je Einwohner

Variablenbezeichnungen beginnen in Zeile 5 – die ersten Zeilen überspringen wir.

> **Was ist hier eine Beobachtung?**

# Ein Tabellenblatt als Vorlage

```r
# Blatt "SH" ohne die ersten 5 Zeilen, nur Spalten 1-7
schulden_sh <- read_xlsx(schulden_pfad, sheet = "SH", skip = 5)[1:7]

# Spalten umbenennen
colnames(schulden_sh) <- c("Regionalschluessel", "Gemeinde", "Verwaltungsform",
                           "Einwohner", "Schulden_gesamt", "Veraenderung_Vorjahr", "Schulden_pro_kopf")

# Merken, aus welchem Blatt die Daten kommen
schulden_sh$Bundesland <- "SH"
```

```r
head(schulden_sh, 4)
```

```
# A tibble: 4 × 8
  Regionalschluessel Gemeinde Verwaltun…¹ Einwohner Schulden…² Veraende…³ Schulden…⁴ Bundesland
  <chr>              <chr>    <chr>       <chr>     <chr>      <chr>      <chr>      <chr>
1 <NA>               <NA>     <NA>        <NA>      <NA>       <NA>       <NA>       SH
2 <NA>               <NA>     <NA>        <NA>      <NA>       <NA>       <NA>       SH
3 <NA>               <NA>     <NA>        <NA>      <NA>       <NA>       <NA>       SH
4 <NA>               <NA>     <NA>        <NA>      Euro       %          Euro       SH
# ℹ abbreviated names: ¹Verwaltungsform, ²Schulden_gesamt, ³Veraenderung_Vorjahr,
#   ⁴Schulden_pro_kopf
```

# ✍️ Jetzt Sie: alle Bundesländer

**Teams von zwei Personen, 20 Minuten:** Code, der **alle Bundesländer** einliest und zu *einem* Datensatz zusammenfügt. Nutzen Sie **Chat AI** und posten Sie ihren Code hier (am Besten vorher prüfen):

- `nrow()` – plausibel für ~11.000 Gemeinden?
- `count(Bundesland)` – sind alle 13 Blätter drin?
- `glimpse()` – welche Spalten sind `<chr>`, obwohl sie Zahlen enthalten?

Posten Sie Ihren Code hier →

20:00

# Bug Hunt 3: Eine typische KI-Antwort

```r
sheet_names <- excel_sheets(schulden_pfad)
sheet_read  <- sheet_names[7:19]

for (s in sheet_read) {
  tmp <- read_xlsx(schulden_pfad, sheet = s, skip = 5)[1:7]
  tmp$Bundesland <- s
  schulden_individuell <- tmp
}
```

**1 Minute:** Läuft ohne Fehler. Welche Zeile? Was steht am Ende in `schulden_individuell`?

Zeile 7 **überschreibt** in jedem Durchlauf und am Ende ist nur Thüringen übrig. `count(Bundesland)` zeigt eine einzige Zeile.

01:00

# Lösung: `map()` statt Schleife

```r
sheet_names <- excel_sheets(schulden_pfad)
sheet_read  <- sheet_names[7:19]           # Blätter 7-19: die Bundesländer

spalten <- c("Regionalschluessel", "Gemeinde", "Verwaltungsform",
             "Einwohner", "Schulden_gesamt", "Veraenderung_Vorjahr", "Schulden_pro_kopf")

schulden_individuell <- sheet_read |>
  set_names() |>                                                  # Namen = Blattnamen
  map(\(s) read_xlsx(schulden_pfad, sheet = s, skip = 5)[1:7] |> set_names(spalten)) |>
  list_rbind(names_to = "Bundesland")                             # zu einem Datensatz
```

```r
schulden_individuell |> count(Bundesland) |> pull(n, name = Bundesland)
```

```
  BB   BW   BY   HE   MV   NI   NW   RP   SH   SL   SN   ST   TH
 556 1188 2846  479  900 1259  478 2620 1306   76  460  288  773
```

# Variablen umformen

```
1 head(schulden_individuell, 4)
```

```
# A tibble: 4 × 8
  Bundesland Regionalschluessel Gemeinde Verwaltun…¹ Einwohner Schulden…² Veraende…³ Schulden…⁴
  <chr>      <chr>              <chr>    <chr>       <chr>     <chr>      <chr>      <chr>
1 SH         <NA>               <NA>     <NA>        <NA>      <NA>       <NA>       <NA>
2 SH         <NA>               <NA>     <NA>        <NA>      <NA>       <NA>       <NA>
3 SH         <NA>               <NA>     <NA>        <NA>      <NA>       <NA>       <NA>
4 SH         <NA>               <NA>     <NA>        <NA>      Euro       %          Euro
# ℹ abbreviated names: ¹Verwaltungsform, ²Schulden_gesamt, ³Veraenderung_Vorjahr,
#   ⁴Schulden_pro_kopf
```

Es gibt noch Probleme:

- Die Werte stehen nicht direkt unter den Variablennamen → `NA` im Regionalschlüssel entfernen
- “Einwohner”, “Schulden_gesamt”, “Schulden_pro_kopf” sind `character` (`<chr>`)

# Warum `character`?

```
1 schulden_individuell[28, c("Gemeinde", "Einwohner", "Schulden_gesamt")]
```

```
# A tibble: 1 × 3
  Gemeinde                   Einwohner Schulden_gesamt
  <chr>                      <chr>     <chr>
1 Amt Burg-St. Michaelisdonn {15 594}  {44 481 073}
```

🔮 **Prognose:** Was liefert `as.numeric("{15 653}")`? Und `as.numeric("15 653")`?

```
1 as.numeric(c("{15 653}", "15 653", "15653"))
```

```
Warning: NAs durch Umwandlung erzeugt
[1]    NA    NA 15653
```

Eine **Warnung**, keine Fehlermeldung. Der Code läuft weiter, aber mit “NA” statt Zahlen. Daher ist es wichtig Warnungen nicht zu ignorieren, sondern diesen nachzugehen.

# Variablen umformen

```r
# Hilfsfunktion: Klammern und Leerzeichen entfernen, dann in Zahl umwandeln
zahl <- function(x) as.numeric(gsub("[[:space:]{}]", "", x))

schulden_bereinigt <- schulden_individuell |>
  filter(!is.na(Regionalschluessel)) |>
  mutate(across(c(Einwohner, Schulden_gesamt, Schulden_pro_kopf), zahl)) |>
  filter(nchar(Regionalschluessel) == 12) |>   # nur echte Gemeinden – Landkreis-, Verwaltungs- und Summenzeilen rau
  mutate(landkreis = str_extract(Regionalschluessel, "^.{5}")) |>
  select(-Veraenderung_Vorjahr) |>
  filter(!is.na(Einwohner))        # manche Einheiten haben keine Einwohnerangabe
```

```r
glimpse(schulden_bereinigt)
```

```
Rows: 10,750
Columns: 8
$ Bundesland         <chr> "SH", "SH", "SH", "SH", "SH", "SH", "SH", "SH", "SH", "SH", "SH", …
$ Regionalschluessel <chr> "010010000000", "010020000000", "010030000000", "010040000000", "0…
$ Gemeinde           <chr> "Flensburg, Stadt", "Kiel, Landeshauptstadt", "Lübeck, Hansestadt"…
$ Verwaltungsform    <chr> "kreisfreie Stadt", "kreisfreie Stadt", "kreisfreie Stadt", "kreis…
$ Einwohner          <dbl> 96081, 252175, 217067, 79640, 12747, 22046, 597, 190, 985, 4057, 6…
$ Schulden_gesamt    <dbl> 475045842, 1302633735, 1323857542, 464688174, 71926757, 65851082, …
$ Schulden_pro_kopf  <dbl> 4944.22, 5165.59, 6098.84, 5834.86, 5642.64, 2986.99, 3411.50, 848…
$ landkreis          <chr> "01001", "01002", "01003", "01004", "01051", "01051", "01051", "01…
```

# Reguläre Ausdrücke

Was machen `gsub("[[:space:]{}]", "", x)` und `str_extract(x, "^.{5}")` hier genau?

| Muster | Bedeutung |
|---|---|
| `[[:space:]]` | ein Leerzeichen (oder Tab) |
| `[{}]` | eine geschweifte Klammer |
| `[[:space:]{}]` | eines von beidem |
| `^.{5}` | die ersten fünf beliebigen Zeichen |
| `^[[:digit:]]+\\s*` | Ziffern am Anfang, dann optionale Leerzeichen |

Lassen Sie ihre Regex von Chat AI schreiben, aber achten Sie in der Prüfung auf die Randfälle (Bug Hunt 1).

# Interne Validität: Schulden pro Kopf

Das Statistische Bundesamt liefert "Schulden je Einwohner" fertig. Wir rechnen nach:

**Wette:** Stimmt `Schulden_gesamt / Einwohner` mit der gelieferten Spalte überein? **(a)** exakt **(b)** bis auf Rundung **(c)** echte Abweichungen

```
schulden_consistency <- schulden_bereinigt |>
  mutate(Schulden_pro_kopf_new = round(Schulden_gesamt / Einwohner, 2),
         differenz = Schulden_pro_kopf - Schulden_pro_kopf_new)

range(schulden_consistency$differenz, na.rm = TRUE)
```

```
[1] 0.00 0.01
```

bis auf 1 Cent, d.h. die Daten sind intern konsistent.

# Auflösung: Ihr Landkreis

```r
schulden_bereinigt |>
  filter(str_detect(Gemeinde, "Ulm, Stadtkreis|Biberach an der Riß")) |>
  select(Gemeinde, Einwohner, Schulden_pro_kopf)
```

```
# A tibble: 2 × 3
  Gemeinde                          Einwohner Schulden_pro_kopf
  <chr>                                 <dbl>             <dbl>
1 Stadt Ulm, Stadtkreis                129317            11334.
2 Große Kreisstadt Biberach an der Riß  34515              932.
```

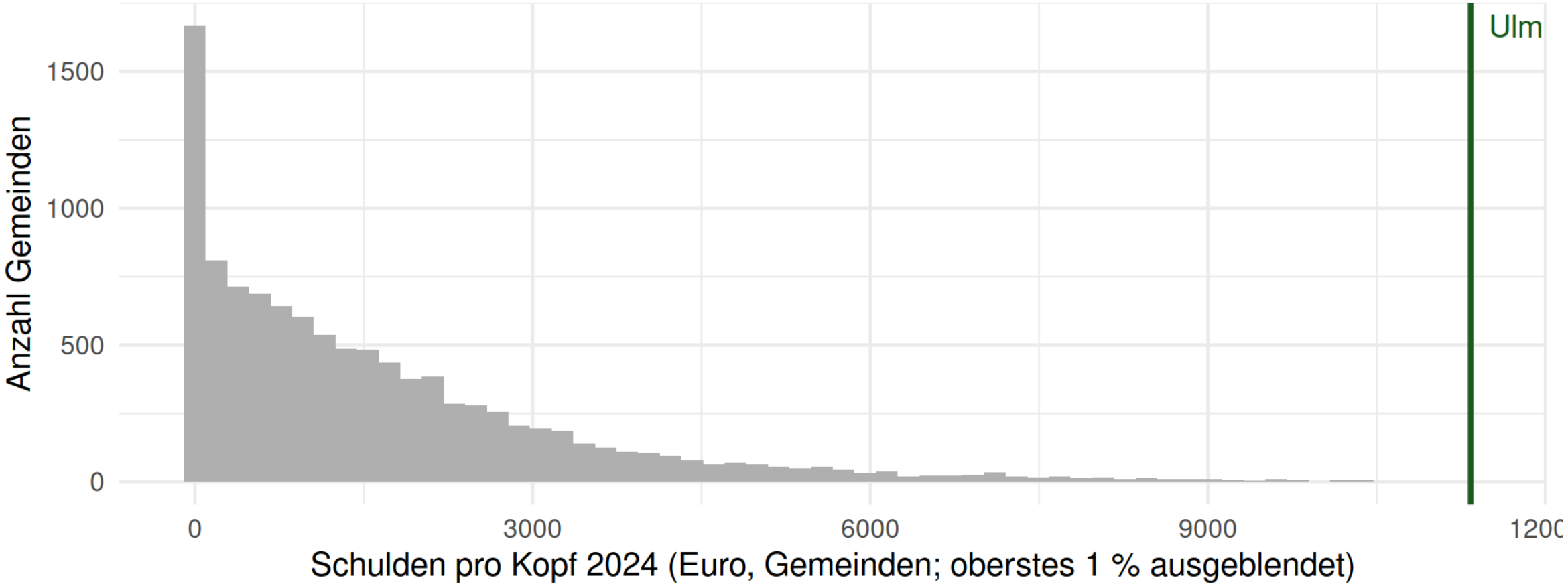
**Jetzt Sie:** Suchen Sie **Ihre** Gemeinde – Suchbegriff unten ersetzen, dann *Run Code* (Teil des Namens genügt):

R Code | Start Over | Run Code

```r
library(dplyr); library(stringr); library(readr)
schulden <- read_csv("data-webr/schulden_gemeinden.csv",
                     col_types = cols(Regionalschluessel = "c"))

schulden |>
  filter(str_detect(Gemeinde, regex("Ulm", ignore_case = TRUE))) |>
  select(Gemeinde, Einwohner, Schulden_pro_kopf)
```

# Auflösung: Ulm im Vergleich




Wer lag mit seiner Schätzung richtig? Was überrascht Sie an der Verteilung?

# Kahoot-Checkpoint 2: Schulden

**kahoot.it** · PIN am Beamer

## 6 Fragen, 4 Minuten

- `as.numeric("{15 653}")` – Warnung oder Fehler?
- `glimpse()` und `gsub()`
- Bug Hunt 3: die Schleife
- Pro-Kopf-Werte richtig aggregieren (Vorgriff!)
- `map()` + `list_rbind()`

Kahoot öffnen →

04:00

# Bruttoinlandsprodukt

Arbeitslose ✔ → Schulden ✔ → **BIP** → Merge → Karte

**Nach diesem Abschnitt können Sie …**

- eine Fehlermeldung von `pivot_longer()` lesen und die Ursache finden,
- Daten vom `wide`- ins `long`-Format bringen,
- wiederholten Code durch eine Funktion ersetzen.

# Informationen zum Bruttoinlandsprodukt

Nach dem Download bei den Statistischen Ämtern interessieren uns:

- Blatt “1.1”: BIP · Blatt “3.1”: Erwerbstätige · Blatt “5”: Einwohner

Außerdem:

- Die ersten vier Zeilen benötigen wir nicht
- Die letzte Zeile ist eine Beschreibung → nur Zeilen behalten, deren `Lfd. Nr.` numerisch ist
- Nicht benötigt: `Lfd. Nr.`, `EU-Code`, `NUTS 1`–`NUTS 3`, `Land`, `Gebietseinheit`

# Einlesen

```r
bip_pfad  <- file.path(datenpfad, "BIP_2024.xlsx")
bip       <- read_xlsx(bip_pfad, sheet = "1.1", skip = 4)
erwerb    <- read_xlsx(bip_pfad, sheet = "3.1", skip = 4)
einwohner <- read_xlsx(bip_pfad, sheet = "5",   skip = 4)
```

```r
bip_wide <- bip |>
  filter(!is.na(suppressWarnings(as.numeric(`Lfd. Nr.`)))) |>          # Beschreibungszeile raus
  select(-c(`Lfd. Nr.`, `EU-Code`, `NUTS 1`, `NUTS 2`, `NUTS 3`, Land, Gebietseinheit)) |>
  rename(Regionalschluessel = `Regional-schlüssel`)
```

# Panel im `wide`-Format

```
1 bip_wide |> select(1:8) |> head(3)
```

```
# A tibble: 3 × 8
  Regionalschluessel `1992`             `1994`             `1995`   `1996` `1997` `1998` `1999`
  <chr>              <chr>              <chr>              <chr>    <chr>  <chr>  <chr>  <chr>
1 08                 256667.12100000001 263604.43400000001 272286.… 27777… 28379… 29287… 30357…
2 081                110814.31200000001 111487.84299999999 114967.… 11666… 12100… 12407… 12845…
3 08111              32842.553999999996 31662.357          32145.6… 32708… 34313… 33567… 35069…
```

Der Datensatz

- ist ein **Panel**: mehrere Jahre für mehrere Landkreise,
- liegt im `wide`-Format vor → nicht `tidy`.

> Was ist hier eine Beobachtung? Welche `tidy`-Bedingung verletzt `bip_wide`?

# Dreimal dasselbe? Dann eine Funktion

Für Erwerbstätige und Einwohner wäre der Code identisch. Dreimal kopieren ist das, was Chat AI gerne tut – und ein Warnsignal:

```
bereinige_vgr <- function(df) {
  df |>
    filter(!is.na(suppressWarnings(as.numeric(`Lfd. Nr.`)))) |>
    select(-c(`Lfd. Nr.`, `EU-Code`, `NUTS 1`, `NUTS 2`, `NUTS 3`, Land, Gebietseinheit)) |>
    rename(Regionalschluessel = `Regional-schlüssel`)
}

erwerb_wide    <- bereinige_vgr(erwerb)
einwohner_wide <- bereinige_vgr(einwohner)
```

Einen Fehler dann nur **einmal** beheben. Fragen Sie Chat AI ruhig: “Fasse die drei identischen Blöcke in eine Funktion zusammen.”

# Fehlermeldungen lesen

```
1 bip_long <- pivot_longer(bip_wide, cols = `1992`:`2024`, names_to = "Jahr", values_to = "bip")
```

```
Error in `pivot_longer()`:
! Can't combine `1992` <character> and `2000` <double>.
```

**1 Minute, zu zweit:** Lesen Sie die Meldung Wort für Wort. Was sagt sie? Welcher *eine* Befehl bestätigt Ihre Vermutung?

01:00

# Fehlermeldungen lesen: Prüfung

```
1 bip_wide |> select(`1992`, `1999`, `2000`, `2024`) |> sapply(class)
```

```
      1992        1999        2000        2024
"character" "character"   "numeric"   "numeric"
```

1992–1999 sind `character`, ab 2000 `double`. `pivot_longer()` kann die Spalten nicht in **eine** Wertespalte stapeln. → Erst umwandeln.



tidyverse-Fehlermeldungen nennen fast immer die betroffenen Spalten **und** die Klassen. Lesen, nicht raten.

# Umwandeln – und die Warnung ernst nehmen

```
1 bip_wide |> mutate(across(`1992`:`1999`, as.numeric)) |> nrow()
```

```
Warning: There were 7 warnings in `mutate()`.
The first warning was:
ℹ In argument: `across(`1992`:`1999`, as.numeric)`.
Caused by warning:
! NAs durch Umwandlung erzeugt
ℹ Run `dplyr::last_dplyr_warnings()` to see the 6 remaining warnings.

[1] 444
```

> Es wird gewarnt, dass NAs erzeugt wurden. **Warum?** Welche Werte sind betroffen?

```
1 bip_wide |>
2   filter(is.na(suppressWarnings(as.numeric(`1992`)))) |>
3   select(Regionalschluessel, `1992`, `1995`)
```

```
# A tibble: 57 × 3
  Regionalschluessel `1992` `1995`
  <chr>              <chr>  <chr>
1 13003              .      .
2 13004              .      .
3 13071              .      .
4 13072              .      .
# ℹ 53 more rows
```

Betroffen sind Zellen mit `-` (kein Wert). Ein `NA` ist hier **inhaltlich richtig**, nach dem Check können wir diese Zeilen umwandeln.

# Umwandeln

```r
zu_zahl <- function(df) df |> mutate(across(`1992`:`1999`, \(x) suppressWarnings(as.numeric(x))))

bip_wide       <- zu_zahl(bip_wide)
erwerb_wide    <- zu_zahl(erwerb_wide)
einwohner_wide <- zu_zahl(einwohner_wide)
```

`suppressWarnings()` erst, **nachdem** wir wissen, was die Warnung bedeutet, ansonsten hätten wir diesen Fehler nie entdeckt.

# Ins `long`-Format

- Einwohner und Erwerbstätige in **1.000 Personen** → × 1.000
- BIP in **Mio. Euro** → × 1.000.000

```r
bip_long <- bip_wide |>
  pivot_longer(cols = matches("^[0-9]{4}$"),        # alle Jahresspalten
               names_to = "Jahr", values_to = "bip") |>
  mutate(Jahr = as.numeric(Jahr),
         bip  = bip * 1e6) |>
  arrange(Jahr)
```

```r
head(bip_long, 3)
```

```
# A tibble: 3 × 3
  Regionalschluessel  Jahr          bip
  <chr>              <dbl>        <dbl>
1 08                  1992 256667121000
2 081                 1992 110814312000
3 08111               1992  32842554000
```

# ✍️ Jetzt Sie: Einwohner ins `long`-Format

**3 Minuten:** Bringen Sie `einwohner_wide` analog zu `bip_long` ins `long`-Format: Spalten `Regionalschluessel`, `Jahr` (numerisch), `einwohner` (in Personen, nicht in Tausend!). Letzte Zeile: das Ergebnis.

Exercise | Start Over | Show Hint | ▷ Run Code

```
einwohner_long <- einwohner_wide |>
  pivot_longer(cols = ______, names_to = "Jahr", values_to = "einwohner") |>
  mutate(Jahr = as.numeric(Jahr),
         einwohner = ______)
einwohner_long
```

03:00

# Lösung als Funktion

Dasselbe für Erwerbstätige, also wieder eine Funktion statt Copy & Paste:

```r
ins_long <- function(df, name, faktor) {
  df |>
    pivot_longer(cols = matches("^[0-9]{4}$"), names_to = "Jahr", values_to = name) |>
    mutate(Jahr = as.numeric(Jahr), across(all_of(name), \(x) x * faktor)) |>
    arrange(Jahr)
}

erwerb_long    <- ins_long(erwerb_wide,    "erw",       1000)
einwohner_long <- ins_long(einwohner_wide, "einwohner", 1000)
```

# Konsistenzcheck: Ihre Aufgabe

**Vergleichen Sie die Einwohnerzahlen aus dem BIP-Datensatz (Blatt 5, Jahr 2024) mit denen aus dem Schulden-Datensatz (2024) auf Landkreisebene.**

Leitfragen:

- Welche Schlüssel-Länge brauchen Sie in beiden Datensätzen?
- Wie groß dürfen die Abweichungen sein (beide beziehen sich auf 2024)?
- Welche Landkreise fallen auf – und warum?



Genau diese Art Check sollten Sie bei **jedem** KI-generierten Datensatz machen: eine Zahl, die Sie aus einer *anderen* Quelle kennen.

# Kartenmaterial und Merge

Arbeitslose ✔ → Schulden ✔ → BIP ✔ → **Merge** → Karte

**Nach diesem Abschnitt können Sie …**

- Datensätze über einen gemeinsamen Schlüssel verbinden (`left_join()`),
- einen fehlerhaften Join an der Zeilenzahl erkennen,
- erklären, warum Pro-Kopf-Werte nicht gemittelt werden dürfen.

# Kartenmaterial hinzufügen

Wir benötigen eine Karte Deutschlands mit Verwaltungsgrenzen als **Shape-File**; einlesen mit dem `sf`-Paket.

Das OpenData-Portal des BKG stellt die Daten kostenlos bereit. Die Dokumentation sollten wir uns **immer zuerst** anschauen.

Das gilt nicht nur für Geodaten, sondern für alle Datenreihen.

Versuchen Sie selbst, die Daten herunterzuladen und über den Regionalschlüssel (ARS) mit BIP, Arbeitslosen und Schulden zu verbinden.

# Datensätze zusammenführen

> **Nun fügen wir die Datensätze zu einem zusammen.**

Vorher:

- Verschuldung auf **Landkreisebene** aggregieren
- BIP auf das Jahr **2024** einschränken (letztes verfügbares Jahr)
- Datensätze anhand des **Regionalschlüssels** verbinden

Die Geodaten speichern wir separat und mergen sie bei Bedarf hinzu.

# 🐛 Bug Hunt 4: Schulden pro Kopf je Landkreis

Chat AI, gefragt nach “Schulden pro Kopf je Landkreis”:

```
1 schulden_kombi_ki <- schulden_bereinigt |>
2   group_by(landkreis) |>
3   summarise(Schulden_pro_kopf_lk = mean(Schulden_pro_kopf))
```

**2 Minuten:** Läuft, liefert plausible Zahlen. Was ist **ökonomisch** falsch an Zeile 3? Konstruieren Sie ein Gegenbeispiel mit zwei Gemeinden.

Stadt: 100.000 Einwohner, 2.000 €/Kopf. Dorf: 500 Einwohner, 0 €/Kopf. Der **Mittelwert** sagt 1.000 €, der Landkreis hat fast 2.000 €/Kopf. Pro-Kopf-Werte müssen **gewichtet** werden: Summen bilden, dann teilen.

02:00

# ✍️ Jetzt Sie: Bug Hunt 4 reparieren

`schulden_bw` enthält alle Gemeinden in Baden-Württemberg. Berechnen Sie `Schulden_pro_kopf_lk` je `landkreis` **richtig** und geben Sie zusätzlich `Einwohner` (Summe) aus.

Exercise | Start Over | Show Hint | ▷ Run Code

```
1 schulden_bw |>
2   group_by(landkreis) |>
3   summarise(Schulden_pro_kopf_lk = mean(Schulden_pro_kopf))
```

# Schulden auf Landkreisebene

```r
schulden_kombi <- schulden_bereinigt |>
  group_by(landkreis) |>
  summarise(Schulden_pro_kopf_lk = sum(Schulden_gesamt) / sum(Einwohner),   # Summen, dann teilen
            Einwohner       = sum(Einwohner),
            Schulden_gesamt = sum(Schulden_gesamt)) |>
  ungroup() |>
  rename(Regionalschluessel = landkreis)
```

```r
# Gegenprobe zur KI-Variante: Ulm, Alb-Donau, Biberach
schulden_bereinigt |>
  filter(landkreis %in% c("08421", "08425", "08426")) |>
  group_by(landkreis) |>
  summarise(mittelwert_ki = mean(Schulden_pro_kopf), gewichtet = sum(Schulden_gesamt) / sum(Einwohner))
```

```
# A tibble: 3 × 3
  landkreis mittelwert_ki gewichtet
  <chr>            <dbl>     <dbl>
1 08421           11334.    11334.
2 08425            1138.     1049.
3 08426             604.      742.
```

# Sonderfall Stadtstaaten

Hamburg und Berlin sind **gleichzeitig** Bundesland und kreisfreie Stadt. In den Arbeitslosendaten heißen sie `02000` und `11000`, in den BIP-Daten?

```
bip_long |> filter(Regionalschluessel %in% c("02", "11", "02000", "11000"), Jahr == 2024)
```

```
# A tibble: 2 × 3
  Regionalschluessel  Jahr          bip
  <chr>              <dbl>        <dbl>
1 11                  2024 208449010000
2 02                  2024 162843535000
```

Nur Länderschlüssel. `nchar(...) == 5` würde sie herauswerfen, und `left_join()` würde **still** `NA` eintragen. Also ergänzen wir eine Kopie mit Kreisschlüssel:

```
stadtstaaten_als_kreis <- function(df) {
  df |> bind_rows(df |>
                  filter(Regionalschluessel %in% c("02", "11")) |>
                  mutate(Regionalschluessel = paste0(Regionalschluessel, "000")))
}

bip_long       <- stadtstaaten_als_kreis(bip_long)
erwerb_long    <- stadtstaaten_als_kreis(erwerb_long)
einwohner_long <- stadtstaaten_als_kreis(einwohner_long)
```

# Erwerbstätige und Einwohner 2024

```r
erwerb_kombi <- erwerb_long |>
  filter(nchar(Regionalschluessel) == 5 & Jahr == 2024) |>
  select(-Jahr)

einwohner_kombi <- einwohner_long |>
  filter(nchar(Regionalschluessel) == 5 & Jahr == 2024) |>
  select(-Jahr)
```

# 🐛 Bug Hunt 5: Der Join

Chat AI, gefragt nach “BIP pro Kopf”:

```
1 bip_kombi_ki <- left_join(bip_long, einwohner_long, by = "Regionalschluessel") |>
2   mutate(bip_pro_kopf = bip / einwohner) |>
3   filter(nchar(Regionalschluessel) == 5 & Jahr == 2024)
```

🔮 **Prognose:** `bip_long` hat 14.272 Zeilen. Wie viele Zeilen hat das Ergebnis von `left_join()` in Zeile 1? Wo liegt der Fehler?

01:00

# Bug Hunt 5: Prüfung

```r
left_join(bip_long, einwohner_long, by = "Regionalschluessel") |> nrow()
```

```
Warning in left_join(bip_long, einwohner_long, by = "Regionalschluessel"): Detected an unexpected many-to-many relationship between `x` and `y`.
ℹ Row 1 of `x` matches multiple rows in `y`.
ℹ Row 1 of `y` matches multiple rows in `x`.
ℹ If a many-to-many relationship is expected, set `relationship = "many-to-many"` to silence
  this warning.
[1] 456704
```

Jedes BIP-Jahr wird mit **jedem** Einwohner-Jahr kombiniert (32 × 32). dplyr warnt vor der *many-to-many*-Beziehung und `filter(Jahr == 2024)` schlägt danach fehl, weil es nur noch `Jahr.x` und `Jahr.y` gibt.

**Faustregel:** Nach jedem `join` die Zeilenzahl prüfen. Bei `left_join` darf sie nicht steigen, sonst fehlt ein Schlüssel.

# ✍️ Jetzt Sie: Bug Hunt 5 reparieren

`bip_long` und `einwohner_long` (nur Landkreise, alle Jahre) sind geladen. Reparieren Sie den Join und berechnen Sie `bip_pro_kopf` für 2024. Die Zeilenzahl von `bip_long` ist `nrow(bip_long)`. Prüfen Sie sie nach dem Join!

Exercise | Start Over | Show Hint | ▷ Run Code

```
1 bip_kombi <- left_join(bip_long, einwohner_long, by = "Regionalschluessel") |>
2   mutate(bip_pro_kopf = bip / einwohner) |>
3   filter(Jahr == 2024)
4 bip_kombi
```

# BIP pro Kopf

```r
bip_kombi <- left_join(bip_long, einwohner_long, by = c("Regionalschluessel", "Jahr")) |>
  mutate(bip_pro_kopf = bip / einwohner) |>
  filter(nchar(Regionalschluessel) == 5 & Jahr == 2024) |>
  select(-c(Jahr, einwohner))
nrow(bip_kombi)
```

```
[1] 400
```

```r
summary(bip_kombi$bip_pro_kopf)
```

```
   Min. 1st Qu.  Median    Mean 3rd Qu.    Max. 
  21481   36301   42721   47951   52178  193293 
```

> Plausibilitätscheck: Durchschnittliches BIP pro Kopf in Deutschland ≈ 48.000 €. Passt die Größenordnung? Was wäre ohne `* 1e6` passiert?

# Alles zusammenführen

```r
gesamtdaten <- alo_landkreis |>                                  # Basis: Arbeitslose je Landkreis
  mutate(bundesland = str_extract(Regionalschluessel, "^.{2}")) |>
  left_join(schulden_kombi, by = "Regionalschluessel") |>
  left_join(bip_kombi,      by = "Regionalschluessel") |>
  left_join(erwerb_kombi,   by = "Regionalschluessel")
```

```r
nrow(gesamtdaten)
```

```
[1] 400
```

```r
colSums(is.na(gesamtdaten))
```

```
Regionalschluessel           total_alo          bundesland Schulden_pro_kopf_lk
                 0                   0                   0                   4
         Einwohner      Schulden_gesamt                 bip         bip_pro_kopf
                 4                   4                   0                   0
               erw
                 0
```

> Deutschland hat 400 Landkreise und kreisfreie Städte. Stimmt `nrow()`? Sind die `NA`s ein Fehler oder eine Eigenschaft der Daten?

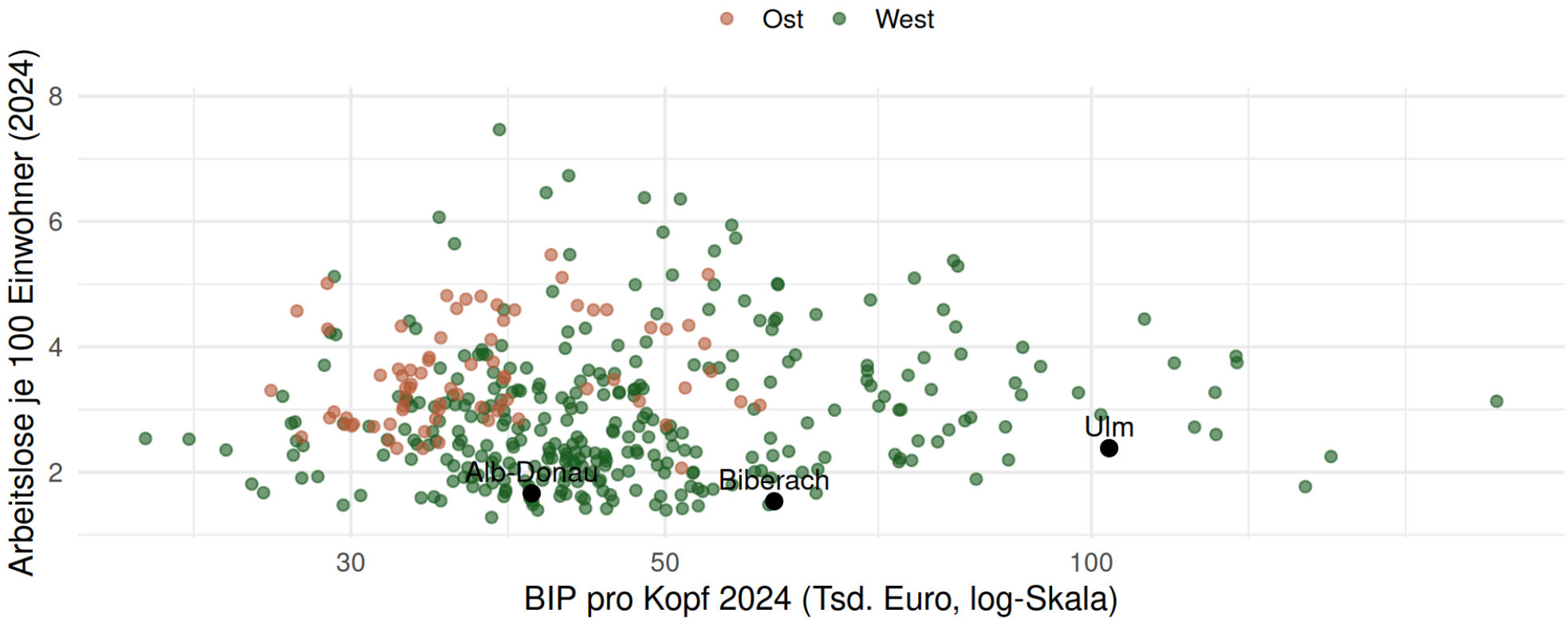
# Ein bekannter Wert

Der letzte Check ist immer derselbe: **eine Zeile, die Sie kennen.**

```r
gesamtdaten |>
  filter(Regionalschluessel %in% c("08421", "08425", "08426", "09775")) |>
  mutate(alo_je_100_ew = 100 * total_alo / Einwohner) |>
  select(Regionalschluessel, total_alo, Einwohner, alo_je_100_ew, Schulden_pro_kopf_lk, bip_pro_kopf)
```

```
# A tibble: 4 × 6
  Regionalschluessel total_alo Einwohner alo_je_100_ew Schulden_pro_kopf_lk bip_pro_kopf
  <chr>                  <dbl>     <dbl>         <dbl>                <dbl>       <dbl>
1 08421                  3083.    129317          2.38               11334.      102934.
2 08425                  3359.    201941          1.66                1049.       40250.
3 08426                  3184.    207290          1.54                 742.       59705.
4 09775                  2945.    182962          1.61                1662.       43641.
```

# Zurück zur Ausgangsfrage




> "Arbeitslose je 100 Einwohner" ist **nicht** die offizielle Quote. Was fehlt im Nenner? (→ Teil 2)

# Code-Review-Checkliste

Bevor Sie Code übernehmen – von Chat AI, aus dem Internet oder von sich selbst:

**1. Läuft er?** Fehlermeldung *lesen*, nicht wegklicken.

**2. Warnungen?** Jede Warnung einmal verstehen, dann erst unterdrücken.

**3. Dimensionen:** `nrow()` vor und nach jedem Schritt. Joins dürfen Zeilen nicht vermehren.

**4. Klassen:** `glimpse()` – Zahlen als `<chr>` sind ein Warnsignal.

**5. Fehlende Werte:** `colSums(is.na())` – und *warum* fehlen sie?

**6. Ein bekannter Wert:** Ihren Landkreis nachschlagen.

**7. Externe Quelle:** eine Summe gegen die amtliche Zahl prüfen.

**8. Verstehe ich jede Zeile?** Wenn nicht: erklären lassen – und die Erklärung prüfen.

Die fünf Bug Hunts hätte jeder dieser Checks gefunden. Keiner davon war eine Fehlermeldung.

# Kahoot-Checkpoint 3: BIP & Merge

**kahoot.it** · PIN am Beamer

## 7 Fragen, 5 Minuten

- Die `pivot_longer()`-Fehlermeldung lesen
- wide → long
- Bug Hunt 5: der Join und die Zeilenzahl
- Einheiten: × 1.000, × 1e6
- 400 Landkreise – und der letzte Check vor der Abgabe

Kahoot öffnen →

05:00

# Übung

# Übungsaufgaben

Laden Sie das durchschnittliche **Arbeitnehmerentgelt pro Arbeitnehmer und Landkreis** herunter und lesen Sie es in R ein. Teams von 4–5 Personen; Case-Study und Chat AI nutzen.

1. Was ist der Unterschied zwischen *Arbeitnehmerentgelt* und *Bruttolöhne und -gehälter*?
2. Tabelle *Bruttolöhne und -gehälter* in R einlesen.
3. Tabelle bereinigen – der Datensatz soll danach `tidy` sein.
4. Konsistenzcheck: Bruttolöhne pro Bundesland aus den Landkreisen berechnen.
5. Mit dem Datensatz auf GitHub (`einkommen.rds`) vergleichen. Stimmen sie überein?
6. Einkommen mit `gesamtdaten` verbinden.

**Abgabe-Check (Checkliste 3, 5, 6, 7)**

`nrow()` vor/nach dem Join · `colSums(is.na())` · Wert für Ulm · Summe BW gegen amtliche Zahl

20:00

# Prompting: Wie fragt man Chat AI?

**Wie sollte ein Prompt aufgebaut sein?**

- **Aufgabe:** Was soll Chat AI tun? Möglichst genau und spezifisch.
- **Instruktionen:** Wie soll es das tun? Persona? Beispiele für den Output? Schritt für Schritt?
- **Kontext:** Was muss Chat AI wissen? (Paket, Datenstruktur, Vorlesung)

**Wenn Sie nicht weiterwissen:**

- “Sag mir, welche Informationen du benötigst, um …”
- “Stelle mir 5 Fragen, die dir helfen, die bestmögliche Antwort zu geben.”

**Komplexe Aufgaben in kleine Unteraufgaben zerlegen** – und nach jeder Unteraufgabe prüfen.

# Prompting: Verifizieren statt vertrauen

- Geben Sie `glimpse()`-Output mit – dann rät Chat AI die Spaltentypen nicht.
- Lassen Sie sich zu jedem Code-Block **einen Check** mitliefern: *“Wie prüfe ich, dass das Ergebnis stimmt?”*
- Fragen Sie: *“Welche stillen Fehler könnte dieser Code haben?”*
- Wenn Sie einen Fehler vermuten: konkret sagen – mit Zeilenzahl oder Output.

Chat AI hat heute fünfmal plausiblen, lauffähigen, falschen Code geliefert. Das wird nicht aufhören. Die Checkliste schon lesen zu können, ist der Unterschied.